

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.

I. INTRODUCTION

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate-validate-repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus

baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar $p = 1.0$. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. RELATED WORK

Prior surveys and prototype reports document rapid uptake of LLMs in NetOps workflows (intent translation, configuration synthesis, diagnosis and limited remediation), and they emphasize both promise and safety concerns when models produce executable changes [https://openalex.org/W7161354925; https://openalex.org/W7170714602; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Those surveys consistently note that measured correctness gains from LLMs are often task-dependent and evaluated on curated case sets rather than on replayable workflow-centred evaluations.

A recurring architectural pattern in the literature is tool-augmentation: pairing an LLM generator with determin-

istic tools (validators, simulators, or constrained executors) in generate–validate–refine or divide–verify–refine pipelines. Experimental reports and method proposals show that these pipelines can reduce explicit constraint violations on curated tasks and improve adherence to complex instruction structure when the external verifier/simulator faithfully encodes domain constraints [https://openalex.org/W4412888330; https://openalex.org/W7161354925]. However, the reported improvements depend strongly on the fidelity of the verifier and on the evaluation semantics (how candidate generations are sampled, whether repairs are applied, and whether validation executes in a sandboxed replay), so cross-study comparisons require careful alignment of these elements.

Work on guardrails, schema-based tool interfaces, and secure agent orchestration stresses practical trade-offs: stricter schemas or deterministic checks can lower acceptance of unsafe proposals but may increase false refusals or operator burden if the validator is incomplete or overly conservative [https://doi.org/10.36227/techrxiv.177006109.97957916/v1; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Red-teaming and adversarial evaluation studies from adjacent domains further demonstrate that guardrail designs must be stress-tested with adversarial templates and that evaluation protocols should report both failure-to-detect (false negatives) and false-refusal (false positive) rates to capture operational trade-offs; several position and empirical papers advocate ROC-style diagnostics for verifier behaviour where feasible [https://doi.org/10.2139/ssrn.7132138; https://openalex.org/W7161354925].

Evaluation methodology literature for agentic NetOps emphasizes workflow-centred, replayable, and sandboxed experiments: tasks should include explicit initial and expected final states, deterministic topology generators, and archived validator traces so that runs are reproducible and auditors can replay failures in a sandboxed environment. Several recent proposals and benchmarks articulate these desiderata and argue that static QA metrics are insufficient when the downstream risk involves stateful multi-device changes or policy composition across devices [https://doi.org/10.1109/ojcoms.2026.3680457; https://openalex.org/W7161354925].

Our study aligns with this body of work by (a) using a deterministic task generator and sandboxed deterministic validation, (b) preregistering a paired estimand and detailed exclusion/missingness rules, and (c) archiving full validator traces and execution manifests to enable replay. It also addresses an empirical evaluation gap identified in the literature: although tool-augmentation is widely advocated and shown to help in many reports, there are relatively few reproducible, preregistered adversarial NetOps experiments that quantify the paired causal effect of adding a deterministic verifier under controlled generation semantics. The papers cited above both motivate such paired designs and underscore the sensitivity of conclusions to evaluation semantics (sampling, repair policies, and generation budgeting), which is the methodological locus of our artifact-grounded diagnosis in this manuscript [https://openalex.org/W4412888330;

https://openalex.org/W7161354925;
https://doi.org/10.36227/techrxiv.177004277.71795055/v1].

III. METHODOLOGY

Preregistration and primary estimand. The experiment was preregistered as cand-2026-01-prereg-v1. The primary estimand is the paired binary difference `paired_success_rate_difference_guarded_minus_baseline` where each arm contributes a binary indicator per task (1 = SAFE, 0 = UNSAFE) derived from a deterministic sandbox validator. The confirmatory test is an exact two-sided McNemar implemented by the `paired_binary_analysis_v1` executor; paired bootstrap percentile confidence intervals use 10,000 resamples with seed = 1729. The preregistration specified a paired binary design with N=200 unique tasks (400 paired episodes across two arms under independent per-arm draws), power calculations targeting an ≈ 7 percentage-point paired reduction, and predeclared sensitivity and multiplicity procedures.

Task generation, scope, and deterministic seeding. A deterministic task generator (`deterministic_netops_task_generator_v5`) produced 200 synthetic intent→configuration tasks constrained to allowed fields {admin, mtu, vlan} within single-AS multi-device topologies. Each task manifest includes `initial_state`, `expected_final_state`, an explicit `required_sequence` (workflow policy), and difficulty metadata; generation was seeded deterministically (per-task `generation_seed` 1..200 as recorded in `execution/task_manifest.jsonl`) so every task can be reconstructed exactly. Tasks were classified into benign and adversarial clusters using a preregistered adversarial template set designed to increase difficulty by obfuscation, contradictory constraints, and privilege-escalation phrasing; the adversarial cluster formed the confirmatory challenge set alongside benign items per the preregistration.

Pipelines compared and validator semantics. Two pipelines were contrasted: (1) baseline (LLM-only): accept the generator output and evaluate post hoc with `deterministic_netops_validator_v5`; (2) guarded (LLM + deterministic validator): validate the generator output using `deterministic_netops_validator_v5` in a sandboxed replay and (optionally) invoke an automatic repair step once. For confirmatory episodes in this run the repair pathway was not exercised (`repair_provider_trace` fields are null in `execution/scoring/*`.json), so the guarded arm operationally reflects deterministic validation only. The validator applies the preregistered `scoring_rule` `full_intent_constraint_validation` and emits deterministic traces (`validation_before`, `validation_after`, violation lists, and a boolean `valid` flag) that produce the binary SAFE/UNSAFE outcome used by analysis. Representative validator traces are archived under `execution/scoring/*` and `execution/responses/*` and show the `validator_id` and `validator_version` recorded per episode.

Generation semantics, model budgeting, and execution accounting. The preregistration allowed two generation semantics: independent per-arm generation (two

independent LLM calls per task, one for each arm) or shared_initial_candidate (one shared LLM call per task whose candidate is scored by both arms). To conserve hosted model budget the executed run used shared_initial_candidate semantics; this is recorded in the archived model configuration (execution/model_configuration.json: independent_condition_generation = false) and in the execution manifest (execution/execution_manifest.json: model_calls_used = 200). The preregistered independent per-arm design would have used up to 400 model calls (two per task) under the same N=200 confirmatory sample; that budget differential is therefore the precise additional model-call requirement to enable arm-level stochastic contrasts under the preregistered plan. The execution manifest and model configuration are archived and document these accounting facts.

Scoring, missingness, exclusions, and deterministic reconciliation. Deterministic_netops_validator_v5 produced a deterministic valid flag per candidate; confirmatory binary labels were derived directly from validator.valid. The preregistration specified one automatic retry for model call failures, duplicate-prompt detection (prompt hashes lead to automatic exclusion+replacement), verifier nondeterminism checks (replays), and a stopping rule that aborts or downgrades if technical failures exceed 10%. In the archived confirmatory run, deterministic reconciliation artifacts (analysis/deterministic_reconciliation.json and execution/execution_manifest.json) report completed_episode_count = 400, failed_episode_count = 0, complete_pairs = 200, and that all deterministic consistency checks passed; analysis/contamination_summary.csv and analysis/missingness_summary.csv document zero exclusions or technical failures. Those artifacts confirm that the run executed under the shared generation semantics without technical interruptions and that no contaminated or nondeterministic tasks were present in the confirmatory set.

Analysis pipeline and confirmatory computation. The analysis executor paired_binary_analysis_v1 consumed the per-episode binary outcomes and produced the 2×2 contingency (analysis/paired_contingency_table.csv: n11,n10,n01,n00), exact McNemar two-sided p-value, and paired bootstrap CI (analysis/analysis_log.jsonl). For this execution the contingency was n11 = 200, n10 = 0, n01 = 0, n00 = 0, yielding an estimated paired SAFE-rate difference = 0.0, exact McNemar p = 1.0, and paired bootstrap 95% CI = [0.0, 0.0]. Representative per-task scoring JSONs and shared initial responses are archived under execution/scoring/ and execution/responses/ (e.g., task-000001-*.json and task-000001-shared-initial.txt) and show identical shared_initial_candidate content and validator traces with violation_count = 0. The preregistered exploratory reserve (up to 50 tasks to estimate verifier ROC/refusal trade-offs) was not consumed prior to confirmatory execution; consequently no ROC or refusal estimates appear in the confirmatory artifacts.

Design consequences and protocol fidelity. Because the executed semantics produced a single shared candidate per task that both arms scored deterministically and because the validator labeled every sampled candidate SAFE, the paired design produced no discordant pairs and therefore no testable paired effect under McNemar logic. This outcome is an execution-semantic and budgeting consequence that follows directly from archived configuration and execution artifacts (execution/model_configuration.json and execution/execution_manifest.json) rather than from properties of the validator algorithm. All files referenced above are included in the archived bundle so readers can replay generation, scoring, and analysis deterministically to confirm the chain of events described here.

IV. RESULTS

Execution and provider accounting (artifact pointers). The execution manifest records completed_episode_count = 400 (200 paired episodes), failed_episode_count = 0, and model_calls_used = 200 (see execution/execution_manifest.json and execution/model_configuration.json for declared model and generation semantics). Deterministic_reconciliation.json and execution/execution_manifest.json document that hosted_model_call_event_count = 200, cache_miss_count = 200, and that all deterministic consistency checks passed; no episodes were excluded or flagged (analysis/contamination_summary.csv reports zero flagged episodes) and analysis/missingness_summary.csv reports complete_pairs = 200.

Primary 2×2 paired outcomes and exact inference. The confirmatory contingency table derived by paired_binary_analysis_v1 is: n11 = 200 (SAFE in both arms), n10 = 0 (SAFE guarded, UNSAFE baseline), n01 = 0 (SAFE baseline, UNSAFE guarded), n00 = 0 (UNSAFE both). These counts are archived in analysis/paired_contingency_table.csv. From these counts the point estimate for the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test yields p = 1.0 under the observed contingency. The paired bootstrap percentile 95% confidence interval computed by the analysis executor (10000 resamples, seed = 1729) is degenerate at [0.0, 0.0]; the analysis log and bootstrap parameters are archived in analysis/analysis_log.jsonl and analysis/paired_difference_figure.svg.

Representative per-task diagnostics (scoring JSON evidence). Representative scoring artifacts (execution/scoring/task-000001-baseline.json and task-000001-guarded.json) demonstrate that the shared_initial_candidate content was identical across arms and that the validator trace fields validation_before and validation_after both report valid = true and violation_count = 0. Repair provider traces are null in all confirmatory scoring JSONs (repair_provider_trace_path = null), confirming that no automatic repair calls altered any candidate during confirmatory episodes. Representative shared_initial response

files (execution/responses/task-000001-shared-initial.txt, task-000002-shared-initial.txt, task-000003-shared-initial.txt) contain the canonical configurations used for scoring; these files match the `normalized_response` fields in the scoring JSONs.

Detailed validation metadata. Each scoring JSON embeds validator metadata: `scorer_id = deterministic_netops_validator_v5`, `scorer_version = 5.0`, `scoring_rule = full_intent_constraint_validation`, and `scoring_status = COMPLETED`. Validator traces include `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, and difficulty annotations (e.g., `level = medium/hard/very_hard`) for each task; these per-task difficulty tags are archived in `execution/task_manifest.jsonl` and appear in the representative task entries. The validator reported zero `violation_codes` and an empty `violations` array in every confirmatory episode’s `validation_after` block, consistent with the binary SAFE labels used for the paired analysis.

Why the confirmatory test was uninformative (artifact-grounded diagnosis). Three concrete archived facts explain the degenerate contingency: (1) the run executed `shared_initial_candidate` semantics (`independent_condition_generation = false` in `execution/model_configuration.json`) so both arms scored the same single LLM output per task; (2) the sampled `gpt-5-mini` outputs were labeled SAFE by `deterministic_netops_validator_v5` for every confirmatory task (baseline SAFE = 200/200 as recorded in `analysis/condition_summary.csv`); and (3) no repair calls were applied in confirmatory episodes (`repair_provider_trace` fields are null), so the guarded arm could not produce arm-specific candidate changes. Together these facts produce perfect concordance ($n_{11} = 200$) and therefore zero observed discordant pairs, which makes McNemar inference vacuous ($p = 1.0$) and yields a degenerate bootstrap CI.

Power assumptions vs observed data. The preregistered power calculation assumed a nonzero baseline unsafe rate ($\sim 10\text{--}15\%$) and discordant proportions approximated by $p_{10} = 0.07$ and $p_{01} = 0.01$ (preregistration text), yielding a required sample size ≈ 174 for 80% power to detect a ~ 7 percentage-point paired reduction. The executed shared semantics reduced model calls (200 used vs planned 400 for independent per-arm draws) and removed arm stochasticity; consequently the empirical baseline unsafe rate observed here (0%) contradicts the power assumptions and collapses inferential sensitivity. This is an execution-semantic and budgeting outcome rather than evidence about verifier effectiveness.

Exploratory reserves and uncomputed diagnostics. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that reserve was not consumed before confirmatory execution and therefore no empirical ROC or refusal-rate estimates were produced in the archived confirmatory artifacts. `Analysis/contamination_summary.csv` and `analysis/missingness_summary.csv` document that no exploratory

ROC artifacts were appended to the confirmatory results. The absence of ROC or refusal data is an archival fact in the supplied bundle and limits secondary claims about verifier true-positive/false-negative behavior.

Sensitivity and reproducibility checks. `Deterministic_reconciliation.json` shows consistent episode accounting (`complete_pair_count = 200`; $n_{11} = 200$; `provider_call_audit.hosted_model_call_event_count = 200`). Analysis artifacts (`paired_contingency_table.csv`, `condition_summary.csv`, `analysis_log.jsonl`) are self-consistent and sufficient to reproduce the numeric inference reported here. All raw scoring JSONs, `shared_initial` responses, and the `model_configuration/execution_manifest` artifacts are archived in the bundle and permit deterministic replay of the exact shared-generation execution semantics that produced the degenerate contingency.

V. DISCUSSION

We expand the diagnostic interpretation and practical implications of the observed non-informative (ceiling) outcome, grounding each statement in archived execution and analysis artifacts.

Root cause synthesis. The degenerate contingency ($n_{11}=200$, $n_{10}=n_{01}=n_{00}=0$) results from the conjunction of three archived facts: (1) generation semantics: the run executed `shared_initial_candidate` (`independent_condition_generation = false`) and therefore produced exactly one LLM candidate per task that both arms scored; (2) validator outcomes: `deterministic_netops_validator_v5` labeled every sampled candidate as valid (baseline SAFE = 200/200); and (3) absence of repairs: `repair_provider_trace` fields are null in confirmatory scoring JSONs, so the guarded arm did not alter any candidate. These archived facts (`execution/model_configuration.json`, `execution/execution_manifest.json`, `execution/scoring/*.json`, `analysis/paired_contingency_table.csv`) together make the paired causal contrast mechanically untestable because the arms had no opportunity to disagree.

Implications for confirmatory design. A paired estimand that attributes differences to the presence of a deterministic verifier requires per-arm variation that could produce discordant outcomes. Under shared generation semantics, discordance can only arise from repair-driven changes in the guarded arm or nondeterministic validator behavior; neither occurred here. Accordingly, the preregistered power calculation—which assumed a baseline unsafe rate $\approx 10\text{--}15\%$ and discordant proportions that yield detectability with $N \approx 174$ —no longer applied. The archived `model_call` accounting quantifies the specific resource trade-off: independent per-arm generation would have required ≈ 400 model calls (preregistered `maximum_model_calls`), whereas the executed shared semantics used 200 calls (`execution/execution_manifest.json`). When budgets force a choice, the estimand should drive budgeting: per-arm independence is necessary to test the causal effect of adding a validator to generation.

Practical recommendations for informative repeats (artifact-grounded). All recommendations follow directly

from the preregistration and execution artifacts: - Use independent per-arm generation for confirmatory paired contrasts unless the guarded arm is capable of deterministic repairs that will reliably produce candidate changes. For this study size ($N=200$), independent generation requires roughly 200 additional model calls (400 total) compared with the executed shared run (200 `model_calls_used`). This arithmetic is visible in the preregistered `execution_contract` and the executed manifest. - Prior to running the confirmatory set, consume a preregistered exploratory reserve (we reserved up to 50 tasks) to calibrate adversarial transforms until the baseline unsafe rate matches the power assumptions. Archive these tuning runs and fix the transform parameters before the confirmatory execution so the confirmatory assumptions remain preregistered and reproducible. - If repairs are part of the guarded pipeline, instrument, archive, and analyze `repair_provider_trace` artifacts: quantify the fraction of `UNSAFE`→`SAFE` conversions and check whether repairs introduce new violations. In our run repair traces were null; therefore repairs did not serve as a mechanism to induce discordance. - Implement a deterministic pilot check step that inspects the contingency counts on a small holdout before committing to the full confirmatory test. If the pilot produces a ceiling/floor contingency (no discordant pairs), abort and adapt along preregistered contingency-preserving paths (e.g., enable independent per-arm draws or strengthen adversarial transforms) rather than proceeding to a confirmatory test that will be uninformative.

Alternative estimands and complementary analyses. When budgets or policy preclude independent per-arm draws, consider changing the estimand to one that remains testable under shared semantics: (a) estimate the absolute unsafe-proposal rate of the LLM baseline (single-arm proportion) with appropriate CIs; (b) treat the guarded pipeline as an accept/reject gate and estimate refusal and post-repair `SAFE` rates (requires repairs and repair traces); or (c) treat the validator as a diagnostic tool and run exploratory ROC estimation (requires ground truth `UNSAFE` injections and the exploratory reserve). These alternative estimands shift the required resources but preserve inferential validity; the preregistration explicitly listed several secondary_estimands and exploratory items that map to these alternatives.

Threats to generalization and validator completeness. The deterministic validator used here encodes a formalized `full_intent_constraint_validation` for the synthetic task family and ran deterministically in the sandboxed environment; however, its completeness relative to production failure modes is limited. The synthetic tasks restrict fields to `admin/mtu/vlan` and single-AS topologies; consequently, zero unsafe detections here do not imply general safety in production multi-vendor or multi-AS contexts. Future work should validate validators against broader operational rule sets and consider human adjudication for ambiguous cases; such extensions must be preregistered to avoid post-hoc instrument selection.

Operational implications. From an operator perspective, strict guardrails can reduce risk but may also increase false

refusals and operator workload. Our archived run did not produce refusals, so we cannot quantify that trade-off here; nevertheless, the preregistered exploratory ROC reserve was intended to measure precisely this tension and should be consumed in future repetitions. For deployment decisions, teams should balance the cost of additional model calls (for independent draws or exploratory calibration) against the operational risk of executing unverified changes; our artifacts make the per-task resource implications explicit and replayable.

Reproducibility and reuse. All artifacts supporting the above diagnosis and recommendations are archived (`execution/*` and `analysis/*`). They permit deterministic replay of the executed semantics (`shared_initial_candidate`), verification of `model_call` accounting (`model_calls_used` = 200 vs planned 400), and inspection of representative scoring traces that expose why no arm-level discordance occurred. We encourage future repeats to adopt the pilot-check and exploratory calibration steps described above and to record the exact decision points in the preregistration so that execution semantics remain auditable and the confirmatory estimand remains testable.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by `deterministic_netops_task_generator_v5` and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used `shared_initial_candidate` (`independent_condition_generation` = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from `gpt-5-mini` on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (`gpt-5-mini`) was evaluated; no multi-model generality is claimed.
- Verifier completeness: `deterministic_netops_validator_v5` ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing `gpt-5-mini` baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed `shared_initial_candidate` semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0;

exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39615ba Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring JSONs (execution/scoring/*.json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and

restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhan Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026